Regenerating the prebuilt binaries

This is the "how the `bin/` folder gets refreshed" doc. Follow it whenever `minai.cpp` has changed in a way you want the prebuilt binaries (for non-programmers downloading from GitHub) to reflect.

If you are Claude, some future version of Claude, or the repo owner reading this six months from now: you will have forgotten how it works. That is fine. Everything you need is below.

When to do this

- After any change to `minai.cpp` that a non-programmer user would care about (new flag, new task, fixed bug, perceptible output change).
- **Not** after pure documentation changes or comment-only edits.

Prerequisites

- A Mac with Xcode Command Line Tools (provides `c++` / `clang++` supporting `-arch arm64 -arch x86_64`). This is needed only for Step 1 (the macOS universal binary). Steps 2-5 work from any machine.
- `curl`, `unzip`, `python3` — all preinstalled on macOS and on any recent Linux.
- Internet access to GitHub and nightly.link.

No GitHub authentication is required; everything works over public read-only APIs.

TL;DR — the four steps

```
# 1. Rebuild the macOS universal binary locally.
c++ -std=c++17 -O2 -arch arm64 -arch x86_64 -o bin/macos-universal/minai minai.cpp

# 2. Push your minai.cpp changes and wait ~1 minute for GitHub Actions
#    to build Linux and Windows binaries. Check https://github.com/essen48183/MinAI/actions

# 3. Grab the artifact IDs from the latest successful run and download via nightly.link.
RUN_ID=$(curl -s "https://api.github.com/repos/essen48183/MinAI/actions/runs?per_page=1&status=success" \
         | python3 -c "import json,sys;print(json.load(sys.stdin)['workflow_runs'][0]['id'])")
curl -s "https://api.github.com/repos/essen48183/MinAI/actions/runs/$RUN_ID/artifacts" \
     | python3 -c "
import json,sys
```

```
for a in json.load(sys.stdin)['artifacts']:
    print(a['name'], a['id'])"
# -> prints something like:
#    minai-linux-x86_64 6483684369
#    minai-windows-x86_64 6483690475

# 4. Plug the IDs into these two URLs, download, extract, chmod, commit.
#    (Replace LINUX_ID and WINDOWS_ID below.)
curl -sL -o /tmp/linux.zip   "https://nightly.link/essen48183/MinAI/actions/artifacts/LINUX_ID.zip"
curl -sL -o /tmp/windows.zip "https://nightly.link/essen48183/MinAI/actions/artifacts/WINDOWS_ID.zip"
unzip -o /tmp/linux.zip   -d bin/linux-x86_64/
unzip -o /tmp/windows.zip -d bin/windows-x86_64/
chmod 755 bin/linux-x86_64/minai bin/windows-x86_64/minai.exe

git add bin/
git commit -m "Refresh prebuilt binaries"
git push
```

Step-by-step, with explanations

Step 1: Build the macOS universal binary (locally on a Mac)

```
c++ -std=c++17 -O2 -arch arm64 -arch x86_64 -o bin/macos-universal/minai minai.cpp
```

The `-arch arm64 -arch x86_64` flags produce a single Mach-O binary that contains *both* Apple Silicon and Intel Mac code. Any Mac since about 2006 can run it.

Verify:

```
file bin/macos-universal/minai
# should print: Mach-O universal binary with 2 architectures: [x86_64] [arm64]
```

This file lives in the repo. Anything you change in `minai.cpp` has to be rebuilt here — CI cannot produce a universal macOS binary because GitHub's macOS runners only give you one architecture per job.

Step 2: Let GitHub Actions build Linux and Windows for you

The workflow at `.github/workflows/build.yml` runs automatically on every push to `main` that touches `minai.cpp`, `CMakeLists.txt`, or the workflow file. It:

- Spins up an `ubuntu-latest` runner and a `windows-latest` runner in parallel
- Runs `cmake -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build --config Release`
- Smoke-tests each resulting binary with `--help` (fails the build if the binary crashes)

- Uploads each as a GitHub Actions *artifact* named `minai-linux-x86_64` and `minai-windows-x86_64`

Typical end-to-end time: about one minute. Watch it at https://github.com/essen48183/MinAI/actions.

Step 3: Find the artifact IDs

GitHub's REST API is public-read on public repositories. No token required. Two endpoints matter:

```
# List of recent workflow runs. Take the first run_id where status==completed and conclusion==success.
curl -s "https://api.github.com/repos/essen48183/MinAI/actions/runs?per_page=5"

# Artifacts produced by a specific run.
curl -s "https://api.github.com/repos/essen48183/MinAI/actions/runs/<RUN_ID>/artifacts"
```

Each artifact has a numeric `id`. Note the IDs for `minai-linux-x86_64` and `minai-windows-x86_64`.

Artifacts expire after 90 days by default. If the run you want is older, simply rerun the workflow manually from the Actions tab (`workflow_dispatch` trigger) — it will produce fresh artifacts.

Step 4: Download via nightly.link (the one non-obvious step)

GitHub's native artifact download URL (`/repos/.../artifacts/ID/zip`) requires a Bearer token with `actions:read` scope, even on public repos. For that reason, downloading artifacts without authentication is normally impossible.

`nightly.link` is a free public proxy, run by Oliver Jumpertz, that transparently mirrors GitHub Actions artifacts from public repositories at an unauthenticated URL:

```
https://nightly.link/<owner>/<repo>/actions/artifacts/<artifact_id>.zip
```

That is the reason this workflow exists at all. Use it directly:

```
curl -sL -o /tmp/linux.zip   "https://nightly.link/essen48183/MinAI/actions/artifacts/LINUX_ID.zip"
curl -sL -o /tmp/windows.zip "https://nightly.link/essen48183/MinAI/actions/artifacts/WINDOWS_ID.zip"
```

Each downloads a zip containing one file (`minai` or `minai.exe`). Unzip in place:

```
unzip -o /tmp/linux.zip   -d bin/linux-x86_64/
unzip -o /tmp/windows.zip -d bin/windows-x86_64/
```

Sanity check:

```
file bin/linux-x86_64/minai        # ELF 64-bit LSB pie executable, x86-64
file bin/windows-x86_64/minai.exe  # PE32+ executable (console) x86-64
```

Step 5: Set permissions and commit

```
chmod 755 bin/linux-x86_64/minai bin/windows-x86_64/minai.exe
git add bin/
git commit -m "Refresh prebuilt binaries"
git push
```

Verify all three binaries are tracked:

```
git ls-files bin/
# expected:
# bin/README.md
# bin/linux-x86_64/minai
# bin/macos-universal/minai
# bin/windows-x86_64/minai.exe
```

Gotchas I hit, so you don't have to

- **The `*.exe` gitignore trap.** The repo's `.gitignore` lists `*.exe` under MSVC leftovers, but it is scoped to `/*.exe` and `build/*.exe`. Don't broaden it — the prebuilt Windows binary at `bin/windows-x86_64/minai.exe` must remain trackable. If you see `git add bin/` silently skip the exe, run `git check-ignore -v bin/windows-x86_64/minai.exe` to see which rule is blocking it.
- **CI didn't run on your push.** The workflow has a `paths:` filter. It only runs when one of `minai.cpp`, `CMakeLists.txt`, or the workflow file itself is in the diff. To force a run on some other change, trigger it manually from the Actions tab (the workflow has `workflow_dispatch:` enabled).
- **nightly.link is third-party.** If it ever disappears, fall back to `gh` CLI: `brew install gh && gh auth login && gh run download <RUN_ID> --repo essen48183/MinAI`. That produces the same artifacts locally.
- **Binary size check.** All three binaries should be tens to a few hundred KB. If a change makes them grow into megabytes, something was accidentally statically linked — investigate before committing. Current sizes for reference: macOS universal ~200 KB, Linux ~63 KB, Windows ~80 KB.
- **macOS Gatekeeper quarantine**: the committed binary works if you build it yourself, but a fresh clone will trigger the quarantine flag on the binary once someone else downloads it. `bin/README.md` covers the one-line `xattr -d com.apple.quarantine ./minai` workaround. No action needed here during regeneration.

The “when this stops scaling” escape hatch: GitHub Releases

Committing binaries to the repo is fine for a teaching project that changes rarely. If MinAI ever starts getting real revisions — say, more than a handful per year — the canonical long-term answer is GitHub Releases. That would mean:

1. Tag a commit: `git tag v1.0 && git push origin v1.0`.
2. Add a release job to `.github/workflows/build.yml` that fires on tag pushes and uploads each built artifact to the release via `actions/upload-release-asset` or `softprops/action-gh-release`.
3. Update `README.md` and `bin/README.md` to point users at https://github.com/essen48183/MinAI/releases instead of at `bin/`.

Releases are the standard open-source pattern: binaries live outside the repo, stay downloadable forever without dependencies on third-party proxies, and are versioned. When it makes sense to switch, the switch is ~15 lines of YAML plus a few doc updates.

Until then, the flow in this document is the right one.